Ders 3 Genişletilmiş Notu: Yazılım Proje Planı, Alternatif Fizibilite ve Risk Kategorileri

Ders 3 Genişletilmiş Notu: Yazılım Proje Planı, Alternatif Fizibilite ve Risk Kategorileri

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Proje Yönetiminin Üç Ayağı ve Etkin Yönetim
- 2. Yazılım Proje Planı: Yaşayan Bir Plan
- 3. Plan Oluşturulurken Yapılan Beş İş
- 4. Sorunun Tanımlanması
- 5. İş Kırılım Yapısı (WBS)
- 6. Sorunun Bilgisayar Desteğiyle Çözüm Olanağı
- 7. Çözüm Yolunun Kararlaştırılması
- 8. Alternatif Fizibilite
- 9. Seçim Sonrası: Red Nedenleri ve Öncelik Listesi
- 10. Geliştirme Sürecinin Planlanması
- 11. Planın En Az İçeriği
- 12. Dersin Kapsamındaki Araştırma Konuları
- 13. Proje Planının Net İçerik Listesi: Kapsam
- 14. Yazılım Geliştirme Süreç Modelleri
- 15. Risk Analizi, Varsayımlar ve Kısıtlar
- 16. Alt Aktivitelerden Kalite Planına
- 17. Yazılım Risk Yönetimi
- 18. Risk Kategorileri
- 19. Aynı Kategorilerle Varsayım ve Kısıtlar

1. Proje Yönetiminin Üç Ayağı ve Etkin Yönetim

Geçen hafta proje yönetiminin ne olduğu, başarı ölçütleri ve neyin proje sayıldığı konuşulmuştu. Hoca bu derste yazılım proje yönetimi için hangi işlemlerin yapıldığına geçti. Özellikle sorunun, yani problemin anlaşılması aşamasında yapılan çalışmalar üzerinde durdu.

Hoca önce tanıdık bir şemayı hatırlattı. Proje yönetimi, ya da bilgi sistemleri proje yönetimi, üç ayak üzerinde durur. Yazılım proje yönetimini başarmak için üç konuda da beceri gerekir:

- Proje yönetimi bilgisi ve deneyimi.
- Yazılım mühendisliği teknikleri, yani bilgisayar mühendisliği bakış açısı ve bilgisi.
- Genel yönetim.

Etkin proje yönetimi için ise insan kaynakları, problem ve süreç incelenmelidir. Üçünün kapsamı şöyledir:

- **Problem:** Problemin tanımlanması. Bu projenin amacı nedir? Alternatif çözümleri nelerdir?
- **İnsan:** Çalışacak kişilerin seçimi ve performans yönetimi.
- **Süreç:** İşin yapılacağı süreç, yani yazılım geliştirme sürecinde gerçekleştirilen işlemler.

> **Ek bilgi (derste anlatılmadı):** Yazılım mühendisliği kaynaklarında, örneğin Pressman'da, etkin proje yönetimi büyük olasılıkla "dört P" ile anlatılır: people (insan), product (ürün), process (süreç) ve project (proje). Hocanın burada saydığı insan, problem ve süreç üçlüsü bu çerçeveye yakındır. Oradaki ürün başlığı da amaç ve kapsamın belirlenmesini içerir.

2. Yazılım Proje Planı: Yaşayan Bir Plan

2.1 Planın Görevi

Yazılım proje planı, bütün rollerin karşılığını bulabildiği bir ortamda hazırlanır. Burada söz konusu olan yazılım mühendisi ya da sistem analisti değil, yazılım proje yöneticisidir. Proje yöneticisi, üstlendiği görev için gereken kaynakları belirler. Görevi nasıl ve hangi sürede tamamlayabileceğini de belirlemek zorundadır.

Plan, yazılım mühendisinin bilgi ve deneyimine dayanan tahminlerle yapılır. Görev yerine getirilirken yeni bilgiler ortaya çıkar. Plan bunlara göre yeniden gözden geçirilir ve üzerinde düzeltmeler yapılır.

> **Hocanın vurgusu:** Bütün proje planlarında olduğu gibi yazılım proje planı da yaşayan bir plan olmalıdır. Uygulamada ve koşullarda değişiklik olduğunda bu değişiklikleri plana da yansıtabilmemiz gerekir.

Bu nedenle mümkün olduğunca dinamik planlama yapmayı sağlayan proje planlama ortamlarında çalışılmalıdır.

2.2 Farklı Formatlar ve Standartlar

Yazılım geliştirme planının genel alt başlıkları vardır. Bunlar genişletilebilir, çünkü farklı formatlar kullanılır. Hoca, kalite dersini alanların standartları bildiğini söyledi. Yazılım standartlarında, geliştirme sürecinin her aşaması için bir standart ve belgeleme formatı tanımlanmıştır. Yazılım mühendisliği dersinde de bu standartlardan söz edilmiştir.

IEEE standartları denince şu örnekler akla gelir: çok kullanılan gereksinim belgesi (SRS) standardı, proje plan standardı ve konfigürasyon doküman standardı. Proje planının içinde neler olması gerektiğini basit tanımların dışında IEEE standartlarından da inceleyebiliriz.

> **Ek bilgi (derste anlatılmadı):** Bu ailedeki numaralar büyük olasılıkla şunlardır: SRS için IEEE 830, yazılım proje yönetim planı için IEEE 1058, konfigürasyon yönetimi planı için IEEE 828. IEEE 830'un yerini sonradan büyük olasılıkla ISO/IEC/IEEE 29148, IEEE 1058'in yerini ise ISO/IEC/IEEE 16326 almıştır.

2.3 Planın Temel İçeriği

Hoca slayttaki plan içeriğini madde madde geçti.

Birinci madde konudur. Etkin proje yönetiminde "problem nedir?" sorusunun karşılığıdır. Projenin amacı, hedefi, ana işlevleri, diğer özellikleri ve geliştirme süreci belirlenip tanımlanır.

İkinci madde kaynaklardır. Yazılım geliştirme projesinde birinci kaynak insandır. İnsan kaynağı nasıl seçilecek, hangi özelliklere ve yeteneklere sahip olmalı? Kullanıcılar ve müşteriler de burada düşünülür, çünkü plan müşterinin yapısına göre de ayarlanabilir. Bunlara bilgisayar donanımı, yazılımı ve zaman eklenir.

Hoca zamanın durumuna ayrıca değindi. Slaytta zaman 2.4 maddesi olarak kaynakların altına alınmıştır. Ama ayrı bir madde, örneğin üçüncü madde olarak da yazılabilir. Hangi seviyeye ne kadar sürede gelineceği orada daha ayrıntılı belirtilir. Çizelgelerin çoğu da zamana bağlıdır.

Risk yönetimi bir sonraki maddedir. Bu projede hangi riskler var? Risk, şu anda var olmayan ama ileride ortaya çıkabilecek problemleri gösterir. Plan, bunlara karşı nasıl önlem alınacağını da belirtir. Konunun üzerinde dersin sonunda ayrıntılı duruldu.

Dördüncü madde giderdir. Ne kadar harcama yapılacak, bütçe nasıl olacak?

Ek olarak çizelgeler yer alır: görev ağı planı, zamanlama çizelgesi ve iş-kaynak çizelgesi.

Bütün bunları yapabilmek için bir iş kırılım yapısı (Work Breakdown Structure, WBS) gerekir. Bu yapı, proje sırasında yapılacak ve alt başlıklara ayrılmış işlerin hiyerarşik olarak seviyelendirilip gösterilmesidir.

2.4 Planın Cevaplaması Gereken Sorular

Yazılım geliştirme planından şu soruların cevabı beklenir:

- Ne yapılacak?
- Ne kadar kaynakla yapılacak?
- Ne kadar zamanda yapılacak?

  Hocanın vurgusu: Plan çok uzun ve çok karmaşık olmamalıdır. Bu soruların cevapları net olarak izlenebilecek bir yapıda olmalıdır.

3. Plan Oluşturulurken Yapılan Beş İş

Az önceki plan içeriği daha alt maddelere bölünüp toparlanırsa, planı oluşturmak için sırayla yapılacak işler ortaya çıkar:

1. Sorunun tanımlanması.
2. Çözüm yolunun kararlaştırılması.
3. Geliştirme sürecinin planlanması.
4. Proje organizasyonu.
5. Proje zamanlaması.

4. Sorunun Tanımlanması

4.1 Müşteri Talebi ve Genel Ürün

Yazılım geliştirme projesinin ilk aşamasında sorun müşterinin istekleri yönünden tanımlanır. Ancak proje bir yazılım evinin genel bir ürünüyse soru değişir. Böyle bir ürüne neden ihtiyaç duyuldu? Ürünün piyasadaki yeri nedir? Kazandıracağı gelir nedir? İşe bunları düşünerek başlamak gerekir. Hoca bundan sonraki anlatımda bir müşteriden talep geldiğini varsaydı.

4.2 Müşterinin Amacı ve Kullanılacak Dil

Hoca örnek olarak bordro yazılımı, envanter ve trafik kontrolünü verdi. Bir bordro yazılımı hazırlanacaksa üzerinde durulacak sorular şunlardır:

- Müşterinin bu yazılımı hazırlatmaktaki amacı ve hedefi nedir?
- Daha önce bunun yerini tutan başka bir yazılımı varsa bununla ilişkili sorunları nelerdir?
- Yeni yazılımdan beklentileri nelerdir?

Müşteriden alınan bilgilere göre plan yeniden gözden geçirilir ve düzeltilir. Yazılım geliştirmeye başlandığında da ilk baştaki hedef göz önünde tutulur. Olay anlaşıldıkça hedeflerde değişiklik olup olmadığı tekrar kontrol edilir.

Sonucu ve müşterinin isteklerini ilgilendirmeyen, yazılım tekniğine ait sorunlara ilk başta girilmez. Müşterinin çözmek istediği sorun, müşterinin de anlayacağı terimlerle tanımlanır.

4.3 Koşullar, Kısıtlar ve Gereksinimler

Böyle bir projede koşullar ve kısıtlar belirlenmelidir. Bugünkü durumda iş hangi koşullarda yapılacak? Konu açısından kısıtlar var mı? Belirli bir müşteriye hazırlanıyorsa o müşteriyle ilişkili kısıtlar var mı? Bunlar tanımlanır ve yeni sistemin amaç ve hedefleri de böylece belirlenmiş olur.

Bir anlamda burada gereksinimler de tanımlanmış olur. **Gereksinim, bir sorunun çözümü için sistemden beklenen yetenektir.** Proje planından sonraki gereksinim analizinde (requirement analysis) bu çalışma çok ayrıntılı olarak yapılacaktır. Planlama aşamasında ise genel düzeyde şunlar tanımlanır:

- Yazılımdan hangi işlevler isteniyor?

- Sistem hangi yeteneklere sahip olmalı?
- Donanım, yazılım ve kullanıcı ara birimleri bakımından beklentiler neler?
- Yazılımın hangi standartta olması isteniyor?

4.4 Kabul Kriterleri ve Kabuller

Burada müşterinin kabul kriterleri de belirlenmelidir. Müşteri sonuçta nasıl bir ürün istiyor? Ayrıca yazılımı geliştirirken yapacağımız bazı kabuller vardır. Bunlar hem olayların işleyişi açısından, yani fonksiyonel olarak, hem de olanaklar açısından belirlenir.

> **Hocanın vurgusu:** Müşterinin kabul kriterlerini bu aşamada net olarak ortaya koyarsak, kabul muayenesi sırasında müşteriyle anlaşmazlığa düşmeden kabul testlerini daha iyi gerçekleştirebiliriz.

4.5 Amaçlara Ulaşma Mekanizması: Dönüm Noktaları

Sorunun tanımlanması aşamasında, amaçlara ulaşma ve gereksinimleri yerine getirme mekanizması da belirlenir. Bunun için yazılım geliştirme sürecinin dönüm noktaları ve kilometre taşları belirlenmelidir. Gereksinim analizi, tasarım, test ve bakım gibi aşamalarda hangi alt işlemlerin yapılacağı belirlenir, ardından zamanlaması yapılır. Bu temel adımların her birine bir kilometre taşı yerleştirilir.

5. İş Kırılım Yapısı (WBS)

Hoca chat üzerinden WBS kısaltmasını yazdı ve bunu çok sık göreceğimizi söyledi. WBS'den kastedilen, yapacağımız iş hangi adımlardan oluşuyorsa onu alt başlıklara bölerek tanımlamaktır. Türkçede iş kırılım yapısı veya hiyerarşik iş listesi gibi adlarla anılır.

Hocanın verdiği örnek numaralı bir listedir:

```
1.0 Analiz
  1.1 Bilgi toplama
  1.2 Modelleme
  1.3 SRS belgesinin hazırlanması
2.0 Tasarım
  2.1 Ön tasarım
  2.2 Ayrıntılı tasarım
```

> **Ders sonrası not:** Hoca 1.3 maddesini "SRS diyagramının hazırlanması" diye söyledi; doğrusu SRS bir diyagram değil, gereksinim belgesidir. Hoca dersin başka yerlerinde de "SRS belgesi" diyor.

Aynı yapı, geçen hafta söylendiği gibi diyagram olarak da gösterilebilir. İş alt parçaları bir grafik altında çizilir. Biçim değişse de önemli olan aynı şeydir: Hedeflenen problemi çözmek için yapılacak işlemlerin hiyerarşik olarak sıralanması.

6. Sorunun Bilgisayar Desteğiyle Çözüm Olanağı

Müşteri yönünden sorun tamamlandıktan sonra yazılım mühendisi şu işleri yapar:

- Sorunun bilgisayar desteğiyle çözüm olanağını belirler.
- Donanım, yazılım ve kullanıcı ana öğelerinin işlevlerini, kısıtlarını ve aralarındaki uyumluluk durumunu belirtir.
- Yazılımın geliştirilmesi ve üretilmesinde ana sistem düzeyindeki amaç, hedef ve gereksinimleri saptar.
- Üretilecek yazılımın sisteme uygunluğunu denetlemekte kullanılacak kabul kriterlerini koyar.

Yani daha planlama aşamasında kullanıcının nasıl bir sistem istediği, sistemin ana fonksiyonları ve bunların nasıl test edileceği belirlenir. Böylece sorunun tanımlanması aşaması tamamlanmış olur.

Hoca bunu geçen haftaki başarı faktörlerine bağladı. Problem doğru anlaşıldı mı? Kapsamı doğru belirtildi mi? Kaynaklar ve zaman tutturuldu mu? Ürün kaliteli olarak gerçekleştirildi mi? Bu başarıyı yakalamak için izlenen adımlar sırayla şunlardır: sorunun tanımlanması, nasıl çözüleceğinin ve kabul kriterlerinin ortaya konması, gereksinimlerin biraz daha açık hale getirilmesi ve çözüm yolunun kararlaştırılması.

7. Çözüm Yolunun Kararlaştırılması

Problem anlaşıldıktan sonra soru şudur: Bu probleme nasıl bir çözüm vermeliyiz? Yazılım geliştirmede karşımıza çok sayıda çözüm yolu çıkar. Hangisi seçilirse daha etkin bir proje yönetimi yapılır ve sonuçta daha iyi bir yazılım ortaya çıkar? Aslında burada yapılan iş alternatif fizibilitedir.

Bu aşamada çözüm yolunun ayrıntılı bir plan halinde gösterilmesi beklenmez. Yalnızca olası çözüm yolları genel olarak açıklanır. Kararda iki grup etken dikkate alınır:

- Kullanıcıların karşılaşabileceği dış faktörler.
- İçerideki yazılım geliştiricilerin bilgisi. Hangi yoldan gidilirse çalışanlar hızlı üretim yapar, hangi yol onların bilgisinin kuvvetli olduğu alanlarla çakışır?

Belirlenen çözüm yolu, tasarımcıya seçim olanağı tanıyan alternatifler halinde ifade edilir. Proje yöneticisi alternatifleri yazılım tasarımcılarıyla iş birliği yaparak oluşturmalıdır. Birden çok seçenek gösterilir ve en uygununun seçilmesi için yapılabilirlik analizi, yani fizibilite çalışması yapılır. Seçilen çözüm yolu, yazılım ürününün tasarımı ve gerçekleştirilmesi için bir çerçeve oluşturur. Bundan sonra bu yola göre hareket edilir.

Genellikle birbirine yakın ama farklı yaklaşımlar karşılaştırılır ve üzerlerinde tartışılarak karar verilir.

> **Hocanın vurgusu:** Ekip halinde çalışmak ve birlikte tartışmak önemlidir. Proje yöneticisinin yazılım geliştirme bilgisi de olmalıdır. Ama bu kararı verirken yazılım mimarı, yazılım tasarımcısı gibi görevlerdeki kişilere danışarak ve onlarla tartışarak karar vermesi daha iyi olur.

Önerilen çözüm yollarının yapılabilirliği, sınırların ve kısıtların belirlenmesiyle saptanır. Bir çözüm yolu ancak sağlanan kaynaklar ve teknoloji düzeyiyle, verilen süre içinde gerçekleştirilebilecekse yapılabilir bulunur.

Hocanın örneği şudur: En iyi çözüm diye bir şey ortaya atılabilir. Ama onu gerçekleştirmek için eğitim alınması gerekiyordur. Bunun için zamanımız var mı? Hepsi birlikte düşünülerek karar verilir. Önerilen çözüm yollarının her birinin yapılabilirliği bu şekilde ortaya konur.

8. Alternatif Fizibilite

8.1 Fizibilitenin Üç Boyutu

Sistem analizi ve yazılım mühendisliği derslerinden bilindiği gibi, fizibilite çalışması tek açıdan yapılmaz. Ekonomik ve teknik olarak ikiye bölünür. Hatta bir de kullanılabilirlik fizibilitesi vardır: Seçilen çözüm kullanıcılara uygun mu?

Alternatif fizibilite, çözüm yolu kararlaştırılırken hangi alternatiflerin olduğunu ve bunlardan hangisinin bize uygun olduğunu bulma çalışmasıdır. Her çözüm alternatifi için üç fizibilite de yapılır. Sonuçlar kıyaslanarak ve modellenerek bir karara varılır.

Boyut	Sorulan soru	Derste geçen ayrıntılar
Teknik fizibilite	Teknik olarak nasıl bir model öneriliyor?	Teknik altyapı, çözüm yolu, yazılım geliştirme süreci ve kullanılacak süreç modeli
Ekonomik fizibilite	Maliyeti ve getirisi nedir?	Her çözümün parasal karşılığı, harcaması, getirisi ve bu ikisinin kıyaslanması

Boyut	Sorulan soru	Derste geçen ayrıntılar
Kullanılabilirlik fizibilitesi	Uç kullanıcıya uygun mu?	Ekran yapıları, ekran geçişleri, seçilen donanımın kullanıcıya uygunluğu

8.2 Teknik Fizibilite ve Süreç Modeli Kararı

Teknik fizibilite yalnızca altyapı seçimi değildir. Geliştirilecek yazılımda nasıl bir altyapı kullanılacağı kadar, nasıl bir süreç modeli kullanılacağı da önemlidir. Süreç modeli kullanıcının ve müşterinin yapısına, geliştiricilerin yaklaşımları bilmesine göre seçilir. Çevik (agile) model mi kullanılacak? Yoksa daha önce yapılmış bir projede konunun akışı biliniyor da waterfall ile mi gidilecek? Çözüm yolu kararlaştırılırken bu da kararlaştırılır.

8.3 Ekonomik Fizibilite

Teknik fizibilite yapılırken ekonomik fizibilite de birlikte düşünülür. Ekonomik fizibilite yalnızca bu yazılım geliştirme işinin ne kadar maliyetle çıkacağı değildir. Yazılımın ne kadar fayda getireceği üzerinde de durulur.

8.4 Kullanılabilirlik Fizibilitesi ve “En İyi Çözüm”

Kullanılabilirlik fizibilitesi, kabul kriterlerini de verdiğimiz planlanan ürünün kullanışlılık olarak uç kullanıcıya uygun olup olmadığını sorar. Hoca bu boyutun gerekçesini şöyle anlattı: Bazen en ideal çözüm, o kurumun çalışanlarına kullanılabilir gelmez. Bu yüzden en iyi çözümün tanımı genişler. En iyi çözüm aynı anda şunları sağlamalıdır:

- Teknik olarak iyi bir çözüm olmalıdır.
- Ekonomik olarak hem getirisi iyi hem gideri dengeli olmalıdır.
- Çıkan ürün ve kullanıldığı ortam, kullanıcılar tarafından kabul görmelidir.

8.5 Alternatif Fizibilite Matrisi

Aradan sonra hoca sınıfa, alternatif fizibilite matrisi oluşturulurken çözüm yollarının hangi bakımlardan değerlendirildiğini sordu.

Soru: En iyi çözüm yoluna karar verirken çözüm yollarını hangi bakımlardan inceleriz? **Cevap:** Bir öğrenci teknik fizibilite, ekonomik fizibilite ve kullanılabilirlik fizibilitesi açısından değerlendirildiğini söyledi. Hoca bunu onayladı.

Hoca matrisi şu sorularla açıkladı. Her çözüm alternatifi için kapsam ne olacak? Teknik olarak nasıl bir model öneriliyor? Bu çözüm modelinin maliyeti nedir? Diğerlerinden farklı olarak yaratacağı bir gelir var mı? Kullanıcıların bu çözüm modeline yaklaşımı nasıl olacak?

Durumu incelemek için karşılaştırma yöntemleri kullanılabilir:

- Olumlu ve olumsuz yönlerin irdelenmesi.
- Zayıf ve kuvvetli yönlerin kıyaslanması.
- Tablolarla karşılaştırma.
- En kötü durum analizi.
- Simülasyon ve model oluşturma. Sistemin nasıl davranacağı kestirilemiyorsa bu yola başvurulur.

9. Seçim Sonrası: Red Nedenleri ve Öncelik Listesi

Fizibilite matrisi oluşturulduktan sonra seçeneklerden biri çözüm yolu olarak seçilir. Seçilmeyenlerin red nedenleri de açıklanır. Böylece geriye dönüp önerilen çözümün neden seçildiği, diğerlerinin neden seçilmediği konusunda elde belge bulunur.

Önerilen çözüm yolunda üretimin öncelik sırası da bir liste halinde belirtilir. Geliştirme sürecinde gereksinimlerde uyumsuzluklar veya darboğazlar fark edilebilir ya da bütçe aşılabilir. Bu nedenlerle bazı ürünlerden vazgeçmek gerektiğinde bu listeden yararlanılır.

> **Hocanın vurgusu:** Öncelik sırası bildirilmezse kararlar daha zayıf alınmış olur.

10. Geliştirme Sürecinin Planlanması

Önce kabaca bir gereksinim analizi yapılmış ve süreç modeline karar verilmiştir. Çevik model mi, waterfall mı, RAD mi seçildi? Geliştirme süreci bu modele uygun olarak planlanır. Geliştirme sürecinin planlanmasında şu işler yapılır:

- **Geliştirme süreci adımlarının belirtilmesi:** Kullanılan modelin adımları yazılır.
- **Yazılım düzeni ve kalite kontrolünün planlanması:** Hoca slaytta Türkçe "yazılım düzeni" yazdığını, bununla yazılım konfigürasyon yönetimini (software configuration management) kastettiğini söyledi. Hangi dönüm noktasında hangi ürün ortaya çıkacak? Bu ürünler kaydedilir. Sonradan üzerlerinde ne zaman değişiklik yapıldığı da kaydedilir.
- **Araç, yöntem ve gösterimlerin kararlaştırılması:** Gereksinim analizi ve tasarımda UML diyagramları mı kullanılacak? UML diyagramlarından hangileri seçilecek? Yalnız modelleme araçları değil, CASE araçlarının kullanılıp kullanılmayacağı da karara bağlanır.
- **Gider tahmini ve görevlendirme:** Gider tahmini için hangi süreç aralığında hangi kişilerin çalışacağı ve maliyetleri belirlenir.
- **Zamanlama programı:** Hangi aralıkta hangi iş yapılacak, hangi olaylar birbirine bağlı olacak?
- **İşletme ve bakım kaynakları:** Sistemin işletilmesi ve bakımı için gereken kaynaklar tahmin edilir.
- **Terminoloji sözlüğü:** Müşteri, problem ve yazılım geliştirici arasında ortak bir terimler sözlüğü tanımlanır.
- **Bilişim kaynaklarının tanımlanması:** Donanım ve yazılım kaynakları ile özellikle insan kaynakları tanımlanır.

Hoca, kabul kriterlerinin, kısıtların, varsayımların ve risklerin belirlenmesinin de planın içinde olduğunu hatırlattı. Plan belgesi için ileride birkaç farklı alternatif vereceğini söyledi.

11. Planın En Az İçeriği

Hoca bu noktada plan içeriğini tekrar gösterip "minimum böyle bir yapıda" bir proje planı hazırlanması gerektiğini söyledi:

- Konu: Proje planında konu tam olarak anlatılır. Geliştirme süreci modeli de burada, istenirse ayrı bir madde olarak daha ayrıntılı yazılır.
- Kaynaklar: İnsan, bilgisayar donanımı, bilgisayar yazılımı araçları ve zaman. Zaman ayrı bir madde olarak da dışarı çıkarılabilir.
- Risk yönetimi.
- Gider tablosu.
- Zaman ve gider tabloları için ek çizelgeler.

Plan, "ne yapılacak?", "ne kadar kaynakla?" ve "ne kadar zamanda?" sorularına cevap vermelidir. Konu birinci soruyu cevaplar. Hoca buna bir ekleme yaptı: Yazılım geliştirilirken kabuller, kısıtlar ve riskler var mı? Bunlar da göz ardı edilmemelidir, çünkü proje planında bunlardan da bahsedilmesi istenecektir. Sonraki derslerde IEEE'nin proje plan standardı üzerinden de geçilebilecek. Bu standart aynı konuları daha açıklamalı ve ayrıntılı olarak nasıl hazırlayacağımızı gösterir.

12. Dersin Kapsamındaki Araştırma Konuları

Hoca öğrencilerin makale önerilerini konuşurken dersin kapsamını da çizdi. Dersin konusu, proje yönetimi konularını yazılım projeleri için uyarlayan ve yazılım geliştirmedeki farklılıkları öne çıkaran konulardır. İlk derste görülen iç içe üç halka burada da geçerlidir: Yazılım mühendisliği için proje yönetimi tekniklerinin ve yönetim biliminin nasıl kullanılacağı. Genel proje yönetimini yazılıma uygulayan örnekler de bu kapsamdadır.

> **Hocanın vurgusu:** Yazılım risk yönetimi, yazılım ölçümü, yazılım projelerinde maliyet tahmini gibi genel konular araştırma konusu olmamalıdır. Bu konular kapsamındaki yeni açılımlar, yeni yaklaşımlar ve yeni araştırma konuları seçilmelidir.

Bu konular için hocanın andığı kaynaklar şunlardır: IEEE Software, Systems and Software, IEEE Transactions on Software Engineering ve IEEE'nin Engineering Management dergisi. Hoca mümkün olduğunca 2015 sonrası yayınlara bakılmasını istedi.

> **Ders sonrası not:** Hoca "Systems and Software" dergisinin IEEE'nin değil başka bir yayınevinin olduğunu söyledi ama adını hatırlayamadı. Bu büyük olasılıkla Elsevier'in Journal of Systems and Software dergisidir.

Derste konuşulan örnek konular, nasıl konuların seçilebileceğini gösterir:

- **Monte Carlo simülasyonuyla proje takvimi yönetimi:** 2020 tarihli bir bilgi teknolojileri sempozyumu bildirisi. Proje iş listesinin Monte Carlo simülasyonu yardımıyla oluşturulması konusunu ele alıyor.
- **Çevik yazılım geliştirme projelerinde proje yöneticisi var mı?** 2016 tarihli bir yayın. Hoca bunun çok tartışılan bir konu olduğunu söyledi.
- **Çevik proje yönetiminin kurumda kabulü (adoption) ve kritik başarı faktörleri:** 2021 tarihli bir derleme (review) makalesi. Bir öğrencinin anlatımına göre makale proje, takım ve kültür başlıklarını birleştirerek ilerliyor. Takımın nasıl yönlendirileceği, kültür ve projenin nasıl birlikte yürütüleceği, verilerin nasıl daha etkin kullanılacağı ve başarı faktörlerinin nasıl ayarlanacağı soruluyor.
- **Yazılım projelerinde risk faktörleri:** 2016 tarihli bir konferans bildirisi. Hoca, bu çalışmanın risk faktörleri konusunda kendi anlatacaklarından farklı şeyler içerdiğini söyledi.

  > **Ders sonrası not:** Öğrenci makalenin yayın yerini "2021 International System Science" diye söyledi. Bu büyük olasılıkla HICSS (Hawaii International Conference on System Sciences) bildirisidir.

Kurumda kabul konusu hocanın özellikle dikkatini çekti. Yazılım geliştirmede birçok işlem yapıyoruz, ama bu işlemler kurumda nasıl kabul görüyor? Hoca, 15-20 yıl önce konfigürasyon yönetiminin kurumda kabulü üzerine bir çalışma yaptığını anlattı. Günümüzde konfigürasyon yönetimi şarttır ve yazılımdaki değişiklikleri sürekli kaydetmek proje yönetim araçlarıyla kolaylaşmıştır. O yıllarda ise daha büyük bir yük getiriyordu. Yazılım kalitesiyle, dolayısıyla şirket kalitesi ve genel kalite yönetimiyle de ilişkiliydi. Hoca aynı sorunun çevik proje yönetimi için de önemli olduğunu düşünüyor.

13. Proje Planının Net İçerik Listesi: Kapsam

Son bölümde hoca, proje planı için daha net bir içerik listesi üzerinden ilerledi. Listeye göre planda önce projenin kapsamı, hedefleri ve sınırları üzerinde durulur. Az önce görülen sorunun tanımlanması ve çözüm yollarının belirlenmesi, kapsamın tam olarak anlaşıldığını görmemizi sağlar.

Hocanın örneği hastane yazılımıdır. "Hastane yazılımı" çok genel bir isimdir. İçeriği nedir?

- Yalnızca randevu sistemi, kayıt sistemi ve doktorların yaptıkları işlemleri raporlayacağı bir sistem mi?
- Uzaktan randevu alma dışında yatan hasta işlemleri de var mı?
- Hastalar için tanı ve tedavi istatistikleri geliştiriliyor mu?
- Yatan hastayla ilişkili malzeme ve kullanım yönetimi var mı?
- Laboratuvar yönetimi dahil mi?

Yazılım genel amaçlı olabilir, ama hedefleri ve sınırları iyice çizilmelidir. Bu sınırlar gereksinimlerin belirlenmesi, modelleme ve simülasyon çalışmalarıyla çizilir.

Burada iki kavram ayrılır. **Uygulama alanı**, örnekte hastane bilgi sisteminin geliştirilmesidir. **Proje alanı** ise şu andaki projemizde sınırların ne olması istendiğidir.

14. Yazılım Geliştirme Süreç Modelleri

İçerik listesindeki ikinci iş, yazılım geliştirme süreç modelinin belirlenmesidir. Süreç modelleri yazılım mühendisliğinin en temel konularındandır. Seçim yapılırken sorulacak soru şudur: Bu modellerden hangisi kapsamımıza, uygulamamıza, müşterimize, var olan süremize ve kaynaklarımıza uygundur?

Soru: Çevik ve waterfall dışında hangi süreç modelleri vardır? **Cevap:** Bir öğrenci V modelini saydı. Bu model waterfall'a benzer, ek olarak doğrulama (verification) birimleri vardır. Aynı öğrenci "salyangoz model" dediği, risk yönetimine (risk management) daha çok odaklanan modeli de hatırladı. Hoca ise RAD (Rapid Application Development) modeli ile prototip modelini ekledi.

> **Ders sonrası not:** Öğrencinin "salyangoz model" dediği, risk odaklı model spiral (sarmal) modeldir. Hoca da hemen ardından spiral modelden söz ediyor.

Hoca, spiral ya da sarmal modelin waterfall ile prototipin birleşimi olduğunu söyledi. Modelleri iki grupta topladı. Bir yanda çok temel olanlar, öbür yanda özel durumlar için geliştirilmiş olanlar vardır:

- **Temel modeller:** Waterfall (çağlayan), V modeli, prototip modeli, spiral (sarmal) model, RAD ve hocanın adını andığı barok modeli.
- **Özel durumlar için geliştirilen modeller:** Unified Process modeli.
- **Çevik yaklaşım altındaki modeller:** XP, Kanban ve günümüzde çok kullanılan Scrum.

> **Ders sonrası not:** Hoca Unified Process'in Rational Rose altında kullanılmak üzere geliştirildiğini söyledi. Doğrusu büyük olasılıkla şudur: Rational Rose bir UML modelleme aracıdır, Unified Process ise bu araçtan bağımsız bir süreç çerçevesidir. İkisinin bağlantısı aynı şirketten, Rational Software'den, çıkmış olmalarıdır.

Hangi modelin uygun olduğu konusunda çok sayıda çalışma vardır.

> **Hocanın vurgusu:** Uygulamamıza hangi modelin uygun olduğunu seçmek önem taşır, çünkü bundan sonraki adımlarımız ona göre olacaktır. En basitinden WBS yapısı buna göre oluşturulur.

15. Risk Analizi, Varsayımlar ve Kısıtlar

Proje planında risk analizi de yapılır. Bu projede riskler nelerdir? Hangileri daha önemlidir? Ortadan kaldırmak için ne yapılmalıdır?

Hoca listeye burada bir ekleme yaptı: Varsayımlar ve kısıtlar. Terim seçimini de açıkladı. "Kabul" denirse kabul kriterleriyle karışabileceği için "varsayım" demeyi tercih etti. Hoca bazı yerlerde kabul sözcüğünü de aynı anlamda kullanmaya devam etti.

Yazılım geliştirme çalışması yapılırken bazı kabullerle ya da bazı sınırlamalarla ilerleriz. Bunlar risk analizindeki maddelere benzer maddeler halinde belirlenir:

- İşlevsellik (fonksiyonelite) açısından.
- Uygulama koşulları açısından.
- Çalışanlar açısından.
- İşletmecilik açısından.

16. Alt Aktivitelerden Kalite Planına

16.1 Alt Aktiviteler ve WBS

Risk analizinden sonra alt aktiviteler, yani WBS yapısı ortaya konur. Projede yapılacak işler ya sayılarla iç içe yazılmış maddeler halinde ya da diyagramlara bölünerek hiyerarşik olarak gösterilir.

Toplam işimiz yazılımın geliştirilmesidir. Alt aktiviteler bunu gereksinim analizi, tasarım, gerçekleştirme ve test gibi başlıklara böler. Bunların da alt başlıkları belirlenir.

Alt aktivitelerin doğru tanımlanabilmesi için süreç modelinin önceden seçilmiş olması gerekir. Model waterfall, spiral, RAD ya da Scrum ise alt aktiviteler her birinde farklı şekilde dizilir. Yani süreç modeline uygun bir WBS ortaya çıkar.

16.2 Zaman Çizelgesi

WBS bir hiyerarşik iş listesi olduğu için zaman çizelgesi bu listenin yanına yazılabilir. Zaman çizelgesi, her alt adımdaki işlerin ne kadar süreceğini gösterir. Önce her işe bir süre verilir.

Sonra aktiviteler arasındaki ilişkilere göre plan yeniden düzenlenir. Biri bittikten sonra diğeri mi başlıyor, ikisi aynı zamanda mı yürüyor? Bu bağlantının tipi her aktivite için belirlenir. Hoca bu konunun üzerinde önümüzdeki hafta duracağını söyledi.

16.3 İş Gücü Belirleme ve Görev Atama

İş gücü belirlemede her alt aktiviteyi hangi tipte ve kaç kişinin yapabileceği belirlenir. Sonra görev atama aşaması gelir. Hoca ikisinin farkını bir örnekle açıkladı:

- **İş gücü belirleme:** "Bu işin sistem analizi çalışması için 15 gün çalışacak 3 tane sistem analistine ihtiyaç vardır."
- **Görev atama:** "Sistem analisti olarak şu isimli kişilerin göreve atanması uygundur."

16.4 Bütçe

Bütçe çalışmasında ne kadar süre, hangi ücret grubundan ve hangi görev seviyesinden kişinin çalışacağı belirlenir. İnsan dışındaki kaynaklar da bütçeye girer:

- Yeni alınması gereken donanım.
- Satın alınması gereken yazılım araçları.
- Yeni bir uygulama projesine başlanıyorsa benzer uygulamaları yerinde görmek için yol bütçesi.
- Malzemeler.
- Alt yükleniciler.

16.5 Proje Kalite Planı

Kalite planında özellikle gözden geçirme takvimi ve gözden geçirmenin yapısı üzerinde durulur. Gözden geçirmeler ne zaman ve hangi aralıklarla yapılacak? Hangi grup yapacak? Ne kadar sürede tamamlanacak? Ayrıca hangi standartların kullanılacağı da kalite planında belirtilir.

16.6 Plandan Sonra SRS

Proje planının akabinde yazılım gereksinim spesifikasyonu (SRS) belgesi hazırlanır. Planlama için yetecek kadar yapılan gereksinim çalışması, SRS'in tam olarak gerçekleştirildiği anlamına gelmez. Proje planının arkasından ayrıca gereksinim analizi aşaması yapılmalıdır. Hoca planın maddelerinin her birini ileride ayrıca ele alacağını söyledi.

17. Yazılım Risk Yönetimi

Varsayım ve kısıtlar benzer alt başlıklarla oluşturulur, ama risk yönetimi farklı bir konudur. Hoca riski, gelecekte değişimle ortaya çıkacak problemleri seçimlerimizle nasıl önleyebileceğimizi anlatan kavram olarak tanımladı. Bu tanımdaki üç sözcük tek tek açıklandı:

- **Gelecek:** Yazılım projesinin şu anda değil, gelecekte kötüye gitmesine neden olabilecek riskler nelerdir?
- **Değişim:** Müşteri gereksinimleri değişebilir. Geliştirme teknolojileri değişebilir. Bilgisayarlarda değişiklik olabilir. Bu değişim nasıl bir süreç içinde olacak?
- **Seçim:** Kullanacağımız yöntemler ve araçlar içinde nasıl seçim yapmalıyız?

Riskin iki özelliği vardır: belirsizlik ve kayıp. Ortaya çıkması için sorulan soru şudur: Şu anda ortada olmayan bir problem, daha sonraki dönemlerde karşımıza çıkar mı? Bu riskler belirlenmelidir.

> **Ek bilgi (derste anlatılmadı):** Gelecek, değişim ve seçim üçlüsü yazılım mühendisliği kitaplarında Robert Charette'in risk tanımı olarak aktarılır. Belirsizlik, riskin gerçekleşip gerçekleşmeyeceğinin kesin olmaması demektir. Kayıp ise risk gerçekleşirse istenmeyen sonuçlar doğması demektir.

18. Risk Kategorileri

Hoca kategorileri iki düzeyde verdi. Genel düzeyde üç kategori vardır:

- **Proje riskleri:** Projenin zamanı ve kaynakları açısından riskler.
- **Teknik riskler:** Tasarım, kuruluş, doğrulama ve bakım problemleri gibi teknik problemler.
- **İşletmecilik riskleri:** Elde edilecek ürünün pazarlama riski, stratejik yönetim, bütçe ve satış desteği riskleri.

Daha geniş alt sınıflar ise şunlardır: ürün büyüklüğü, işletmecilik, müşteriyle ilişkili, süreç, teknoloji, geliştirme ortamı ve personel riskleri. Hoca geliştirme ortamı riskleri için ayrı bir soru listesi anlatmadı. Teknoloji başlığına karşılık gelen soruları ise "teknik riskler" adıyla geçti.

> **Ek bilgi (derste anlatılmadı):** Bu alt sınıflar, Pressman'ın kitabındaki risk kontrol listesine çok benzer. Orada başlıklar ürün büyüklüğü, iş etkisi, müşteri özellikleri, süreç tanımı, geliştirme ortamı, üretilecek teknoloji, personel sayısı ve deneyimidir. Liste, her projede aynı sorular sorularak bilinen ve öngörülebilir risklerin bulunmasına yarar.

18.1 Ürün Büyüklüğü Riskleri

Önce ürünün tahmin edilen büyüklüğü bulunur. Bu büyüklük satır sayısı (line of code) ya da function point olarak ifade edilir. Program ve dosya sayısı da kullanılır. Ardından şu sorular sorulur:

- Önceki ürünlerimizden daha büyük bir ürün mü? Ne kadar büyük?
- Ürünün yarattığı ve kullandığı veri tabanı ne kadar büyük? Şimdiye kadar hiç yönetmediğimiz bir büyüklük mü?
- Yazılım ürününü kaç kişi kullanıyor?
- Yeniden kullanılan yazılım miktarı ne kadar?

18.2 İşletmecilik Riskleri

- Bu ürün şirket geliri üzerinde ne kadar etkili?
- Dağıtım tarihi uygun mu?
- Bu ürünle ilişkili, birlikte çalışan başka sistemler var mı?
- Son kullanıcıların düzeyi nedir?
- Geç teslim edersek ne kadar maliyet getirir? Anlaşma nasıl yapılmış, dağıtım gecikirse ne kadar zarar ederiz?

Hoca ilk soruyu bir örnekle açıkladı. Geliştirilen yazılımın sahibi, hayata bu projeyle başlayan bir start-up şirket olsun. Proje başarısız olursa şirketin kuruluşu bile tamamlanamaz. Bu yüzden ürünün şirket geliri üzerindeki etkisi işletmecilik riskinde önemlidir.

18.3 Müşteriyle İlişkili Riskler

Bütün müşteriler eşit değildir. Gereksinimleri ve kişilikleri farklıdır. Bu nedenle farklı müşteriler için riskleri belirleyen bir kontrol listesi oluşturulabilir:

- Bu müşteriyle daha önce çalışıldı mı?
- Müşterinin gereksinimleri hakkında somut fikri var mı?
- Müşteri proje ara kontrollerine ve gözden geçirme çalışmalarına katılmaya istekli mi?
- Müşteri ürün alanında teknik olarak bilgili mi?
- Yazılım geliştirme alanında bilgili mi? Yapılan işi anlıyor mu?

> **Hocanın vurgusu:** Müşterinin ara kontrollere ve gözden geçirmelere katılmaya istekli olması önemlidir. Bu yüzden çevik projelerde müşteri temsilcisi ekibin bir üyesi gibidir. Waterfall veya başka bir modelle geliştirilen projede de müşteriden bir temsilci ya da grup belirlenip bu kontrollerin düzenli yapılması sağlanabilir.

“Bu müşteriyle çalışıldı mı?” sorusunun cevabı iki yöne de çıkabilir. Müşterinin yapısının bilinmesi genellikle risk oluşturmaz, hatta daha iyidir. Ama bazen o müşteriyle önceden çalışmışızdır ve zor çalışıldığını biliriz. Bu durumda o müşteriyle çalışmak başlı başına risktir.

18.4 Süreç Riskleri

Yazılım süreci iyi tanımlanmamışsa, analiz, tasarım ve test rastgele yapılıyorsa proje risk altındadır. Bu amaçla Software Engineering Institute’un hazırladığı süreçle ilgili sorular şunlardır:

- Yazılım sürecinin yazılı tanımı geliştirildi mi? Çalışanlar bu dokümanı projede kullanmaya istekli mi?
- Yazılım süreç tanımı diğer projelerde kullanıldı mı?
- Yöneticiler ve teknik personel için yazılım mühendisliği eğitimleri verildi mi?
- Standartlar biliniyor mu?
- Gözden geçirme sonuçları belgeleniyor mu?
- Konfigürasyon yönetimi yapılıyor mu?

Bu sorulara ne kadar çok “evet” denirse süreç o kadar iyi yönetiliyor demektir.

> **Ders sonrası not:** Altyazıda “Software Engineer Enstitü” geçiyor. Bu büyük olasılıkla Carnegie Mellon Üniversitesi bünyesindeki Software Engineering Institute’tur (SEI).

18.5 Teknik Riskler

Bu sorular, yazılım geliştirme sırasında kullanılan teknik kararları ve yapıları yoklar:

- Müşteri ile geliştirici arasındaki iletişime yardımcı olan uygulama spesifikasyon teknikleri kullanılıyor mu?
- Analiz için belirli bir yöntem kullanılıyor mu?
- Kod için belgeleme ve kod içi açıklama var mı?
- Test case tasarımı için bir yöntem kullanılıyor mu?
- Analiz ve tasarım sürecinde yazılım araçları kullanılıyor mu?
- Prototip oluşturuluyor mu? Bunun için bir araç kullanılıyor mu?

18.6 Personel Riskleri

Personel riski en fazla karşılaşılan risktir. Sorular şunlardır:

- İnsanlar yeteneklerine göre mi kullanılıyor?
- Yeterli çalışan var mı?
- Personel çalışma süresinin ne kadarını bu projeye ayırıyor?
- Verilen işler çalışanların beklentilerini karşılıyor mu?
- Personel gerekli eğitimi aldı mı?

Bu soruların çoğunun cevabı “hayır” ise personel riski vardır.

19. Aynı Kategorilerle Varsayım ve Kısıtlar

Hoca bu listelerin yalnız risklere özgü olmadığını söyledi. Personelle, teknik yapıyla, müşteriyle ve işletmecilikle ilişkili varsayımlar ve kısıtlar da aynı başlıklarla ele alınabilir. Çünkü riskler, kısıtlar ve kabuller aynı yazılım geliştirme işiyle ilişkilidir ve aynı kategorilerden etkilenir. Bu derste kategoriler riskler için kullanıldı, ama diğer ikisi için de kullanılabilir.

Hoca sonra proje planında gelinen noktayı özetledi. Proje kapsamı belirlendi. Yazılım geliştirme süreç modeli seçildi. Varsayım ve kısıtlar ile proje risk analizi üzerinde duruldu. Bunlar ürün büyüklüğü, süreç durumu, müşteriyle ilişkiler ve işletmecilik gibi kavramlar çerçevesinde değerlendirilebilir. Alt aktiviteler WBS çerçevesinde belirlenir, her biri için zaman çizelgesi verilir ve aktiviteler arası ilişkiler tanımlanmaya başlanır.

Dersin Toparlaması

Bu derste yazılım proje planının ne olduğu ve nasıl oluşturulduğu görüldü. Plan, tahminlere dayanan ve koşullar değiştikçe güncellenen yaşayan bir belgedir. “Ne yapılacak, ne kadar kaynakla, ne kadar zamanda?” sorularını cevaplar. Plana giden yol sorunun müşteri diliyle tanımlanmasıyla başlar. Kabul kriterleri bu aşamada konur. Sonra teknik, ekonomik ve kullanılabilirlik boyutlarını içeren alternatif fizibilite ile çözüm yolu seçilir. Red nedenleri ve üretimin öncelik sırası belgelenir. Süreç modeli de bu kararın parçasıdır, çünkü WBS doğrudan ona bağlıdır.

Dersin son kısmında proje planının net içerik listesi ve risk kategorileri işlendi. Ürün büyüklüğü, işletmecilik, müşteri, süreç, teknik ve personel başlıklarındaki sorular hem riskleri hem varsayım ve kısıtları belirlemek için kullanılabilir. Hoca önümüzdeki hafta risk analizine ve varsayım-kısıt konusuna devam edeceğini, zaman çizelgesi ve aktiviteler arası ilişkiler üzerinde de duracağını söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.